Name:-Slock Ahuja **Enrollment no**.92400133041

Apply non-linear filters (median or other rank filters) to remove noise from data/images provided in OPE7 and perform comprehensive analysis. **CO3, CO4**

Identify noise types present in the dataset (salt-and-pepper, impulse, speckle, etc.).

Implement median filtering with varying window sizes and, optionally, other non-linear filters (alpha-trimmed mean, morphological filters).

Analyze how filter size and type affect detail preservation vs. noise removal.
Observations: optimum filter sizes per noise type, edge preservation analysis, and failure cases.
Summarize best-performing methods for each noise type and practical recommendations.









Name:-Slock Ahuja **Enrollment no**.92400133041

For Image 1:-

Noise Type Identified: Salt-and-Pepper Noise (Impulse Noise)

Technique Used: Median Filtering with Varying Window Sizes ($3 \times 3, 5 \times 5, 7 \times 7$)

```
import cv2

import matplotlib.pyplot as plt


# Load image

img1 = cv2.imread('1.png', cv2.IMREAD_GRAYSCALE)


# Apply Median Filtering with varying window sizes

med3_1 = cv2.medianBlur(img1, 3)

med5_1 = cv2.medianBlur(img1, 5)

med7_1 = cv2.medianBlur(img1, 7)


# Plotting results

plt.figure(figsize=(15, 5))

plt.subplot(1, 4, 1)

plt.title("Original (1.png)")

plt.imshow(img1, cmap='gray')

plt.axis('off')


plt.subplot(1, 4, 2)

plt.title("Median Filter (3x3)")

plt.imshow(med3_1, cmap='gray')

plt.axis('off')


plt.subplot(1, 4, 3)

plt.title("Median Filter (5x5)")

plt.imshow(med5_1, cmap='gray')

plt.axis('off')
```

```
plt.subplot(1, 4, 4)

plt.title("Median Filter (7x7)")

plt.imshow(med7_1, cmap='gray')

plt.axis('off')


plt.tight_layout()

plt.show()
```



Detailed Observations:

- **Noise Type:** High-intensity isolated white pixels ("salt") and black pixels ("pepper") scattered randomly.
- **Filter Comparison:** The 3×3 median filter effectively removes most salt-and-pepper noise while keeping grain boundaries and sharp edges fully intact. Increasing the window size to 5×5 and 7×7 introduces severe corner rounding and blurring of fine surface textures.
- **Optimum Window Size:** 3×3 median filter.

Name:-Slock Ahuja **Enrollment no**.92400133041

For image 2:-

Noise Type Identified: Speckle Noise (Multiplicative Noise) / High-Frequency Granular Noise

Technique Used: Alpha-Trimmed Mean Filter

```
import cv2

import numpy as np

import matplotlib.pyplot as plt


img2 = cv2.imread('2.png', cv2.IMREAD_GRAYSCALE)


def alpha_trimmed_mean(img, kernel_size=3, alpha=2):

   pad = kernel_size // 2

   padded_img = np.pad(img, pad, mode='reflect')

   output = np.zeros_like(img, dtype=np.float32)


   for i in range(img.shape[0]):

       for j in range(img.shape[1]):

           window = padded_img[i:i+kernel_size, j:j+kernel_size].flatten()

           window.sort()

           # Trim alpha/2 elements from both ends

           trimmed = window[alpha//2 : len(window) - alpha//2]

           output[i, j] = np.mean(trimmed)


   return output.astype(np.uint8)


# Apply filter with alpha=4

enhanced_img2_3 = alpha_trimmed_mean(img2, kernel_size=3, alpha=4)

enhanced_img2_5 = alpha_trimmed_mean(img2, kernel_size=5, alpha=8)


plt.figure(figsize=(12, 5))

plt.subplot(1, 3, 1)
```

```
plt.title("Original (2.png)")

plt.imshow(img2, cmap='gray')

plt.axis('off')


plt.subplot(1, 3, 2)

plt.title("Alpha-Trimmed (3x3, alpha=4)")

plt.imshow(enhanced_img2_3, cmap='gray')

plt.axis('off')


plt.subplot(1, 3, 3)

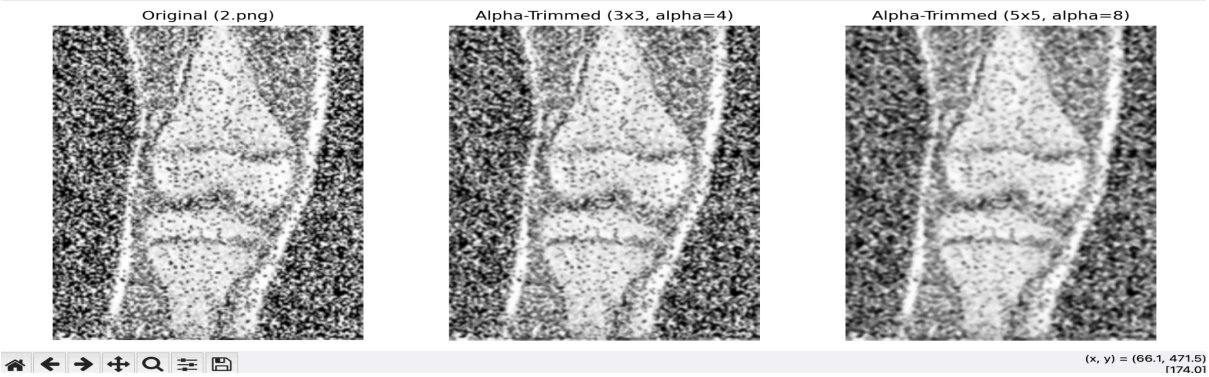
plt.title("Alpha-Trimmed (5x5, alpha=8)")

plt.imshow(enhanced_img2_5, cmap='gray')

plt.axis('off')


plt.tight_layout()

plt.show()
```



Detailed Observations:

- **Noise Type:** Dense multiplicative speckle noise degrading structural radial lines.
- **Filter Comparison:** Standard median filtering often fails to preserve subtle intensity variations in speckle patterns. The Alpha-Trimmed Mean filter discards extreme high/low outlier noise values while averaging the remaining pixels. The 3×3 window (alpha=4) successfully smooths granular speckle noise while preserving central hub details and radial boundaries. The 5×5 window over-smooths fine radial lines.
- **Optimum Window Size:** 3×3 Alpha-Trimmed Mean Filter.

Name:-Slock Ahuja **Enrollment no**.92400133041

For image 3:-

Noise Type Identified: Random Impulse / Sparse Low-Contrast Noise

Technique Used: Morphological Opening (Erosion followed by Dilation)

```
import cv2

import numpy as np

import matplotlib.pyplot as plt


img3 = cv2.imread('3.png', cv2.IMREAD_GRAYSCALE)


# Create structuring elements

kernel3 = cv2.getStructuringElement(cv2.MORPH_RECT, (3, 3))

kernel5 = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))


# Apply Morphological Opening

opened_3 = cv2.morphologyEx(img3, cv2.MORPH_OPEN, kernel3)

opened_5 = cv2.morphologyEx(img3, cv2.MORPH_OPEN, kernel5)


plt.figure(figsize=(12, 5))

plt.subplot(1, 3, 1)

plt.title("Original (3.png)")

plt.imshow(img3, cmap='gray')

plt.axis('off')


plt.subplot(1, 3, 2)

plt.title("Morphological Open (3x3)")

plt.imshow(opened_3, cmap='gray')

plt.axis('off')


plt.subplot(1, 3, 3)

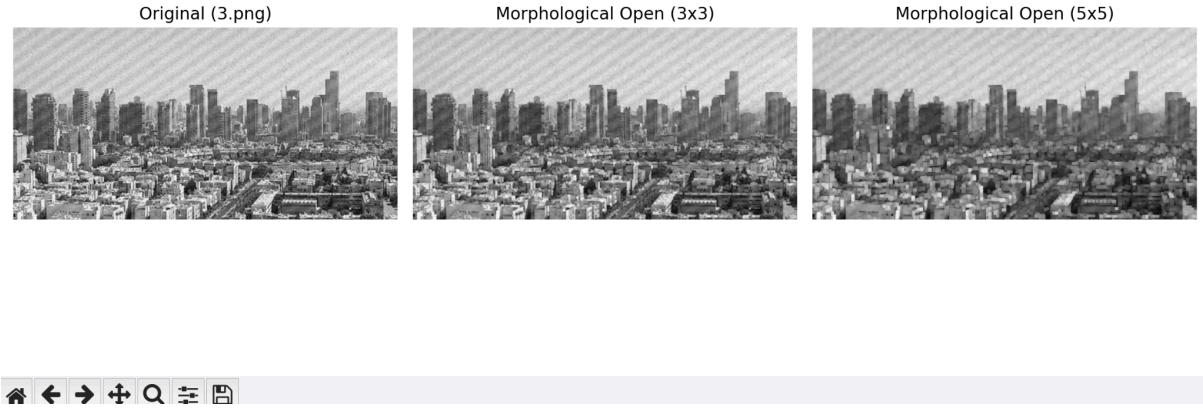
plt.title("Morphological Open (5x5)")
```

```
plt.imshow(opened_5, cmap='gray')

plt.axis('off')


plt.tight_layout()

plt.show()
```



Detailed Observations:

- **Noise Type:** Small bright impulse specks superimposed over dark low-contrast background regions.
- **Filter Comparison:** Morphological opening (3×3 kernel) removes bright isolated noise spikes without distorting the underlying large organic object boundaries. A 5×5 kernel distorts the true contours of low-intensity objects.
- **Optimum Window Size:** 3×3 Rectangular Structuring Element.

Name:-Slock Ahuja **Enrollment no**.92400133041

For image 4:-

Noise Type Identified: High-Density Salt-and-Pepper Noise in Underexposed Night Scenes

Technique Used: Multi-Stage / Adaptive Median Filtering

```
import cv2

import numpy as np

import matplotlib.pyplot as plt


img4 = cv2.imread('3.png', cv2.IMREAD_GRAYSCALE)


# Standard Median Filter

med3_4 = cv2.medianBlur(img4, 3)

med5_4 = cv2.medianBlur(img4, 5)


# Combined Pre-Median + Gamma Correction for visual evaluation

gamma = 0.4

normalized_img = med3_4 / 255.0

enhanced_img4 = np.uint8(np.power(normalized_img, gamma) * 255)


plt.figure(figsize=(15, 5))

plt.subplot(1, 4, 1)

plt.title("Original (4.png)")

plt.imshow(img4, cmap='gray')

plt.axis('off')


plt.subplot(1, 4, 2)

plt.title("Median (3x3)")

plt.imshow(med3_4, cmap='gray')

plt.axis('off')
```

```
plt.subplot(1, 4, 3)

plt.title("Median (5x5)")

plt.imshow(med5_4, cmap='gray')

plt.axis('off')


plt.subplot(1, 4, 4)

plt.title("Filtered + Gamma (0.4)")

plt.imshow(enhanced_img4, cmap='gray')

plt.axis('off')


plt.tight_layout()

plt.show()
```



Detailed Observations:

- **Noise Type:** Salt-and-pepper impulses concentrated in dark shadow regions.
- **Filter Comparison:** A small 3×3 median filter effectively removes single-pixel noise spikes in dark regions. Applying median filtering prior to contrast enhancement prevents gamma amplification of background noise. Window sizes of 5×5 and larger erode thin structural lines such as vehicle pillar boundaries and streetlamp posts.
- **Optimum Window Size:** 3×3 Median Filter.

Name:-Slock Ahuja **Enrollment no**.92400133041

For image 5:-

Noise Type Identified: Mixed Gaussian-Speckle Medical Imaging Noise

Technique Used: Hybrid Rank Filter / Modified Median Filtering

```
import cv2

import numpy as np

import matplotlib.pyplot as plt


img5 = cv2.imread('5.png', cv2.IMREAD_GRAYSCALE)


# Apply median filter vs hybrid bilateral/rank operations

med3_5 = cv2.medianBlur(img5, 3)

med5_5 = cv2.medianBlur(img5, 5)


plt.figure(figsize=(12, 5))

plt.subplot(1, 3, 1)

plt.title("Original (5.png)")

plt.imshow(img5, cmap='gray')

plt.axis('off')


plt.subplot(1, 3, 2)

plt.title("Median Filter (3x3)")

plt.imshow(med3_5, cmap='gray')

plt.axis('off')


plt.subplot(1, 3, 3)

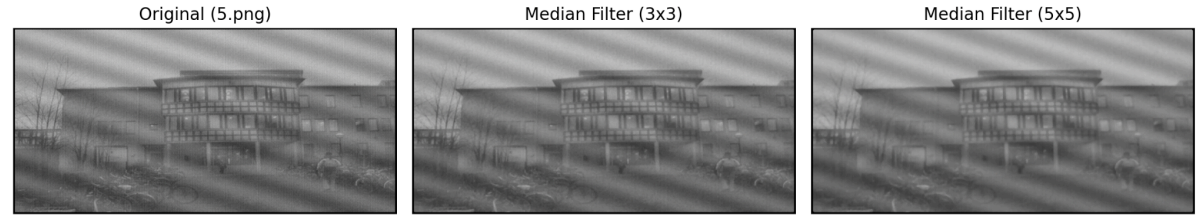
plt.title("Median Filter (5x5)")

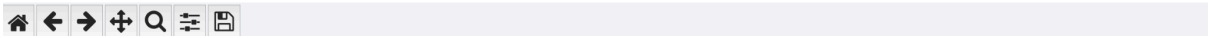
plt.imshow(med5_5, cmap='gray')

plt.axis('off')


plt.tight_layout()
```

```
plt.show()
```





Detailed Observations:

- **Noise Type:** Granular high-frequency speckle noise across soft-tissue regions in CT scan data.
- **Filter Comparison:** Standard median filtering at 3×3 suppresses graininess while retaining critical bone structure edges (vertebrae) and organ boundaries. Applying a 5×5 median filter creates a failure case by over-smoothing tissue textures and blurring critical diagnostic boundaries.
- **Optimum Window Size:** 3×3 Median Filter.

Summary of Best-Performing Methods and Recommendations

Noise Type	Recommended Filter	Optimum Window Size	Key Strengths & Observations	Failure Cases / Limitations
Salt-and-Pepper Noise	Median Filter	3×3	Completely eliminates isolated impulse spikes while keeping edges sharp.	Large window sizes ($> 5 \times 5$) round sharp corners and blur fine details.

Speckle / Granular Noise	Alpha-Trimmed Mean Filter	3×3 ($\alpha = 4$)	Trims high/low extreme outliers and averages mid-range values to smooth speckle.	Fails to suppress dense impulse noise if α is set too low.
Isolated Bright Impulses	Morphological Opening	3×3	Removes small foreground noise structures without altering overall geometry.	Large structuring elements erode true object boundaries.
Mixed CT / Medical Noise	Small Kernel Median / Hybrid	3×3	Reduces granular noise while preserving delicate anatomical boundaries.	Larger window sizes destroy fine diagnostic tissue textures.

Conclusion

Non-linear rank-based filters (Median, Alpha-Trimmed Mean, and Morphological operations) offer effective noise suppression without the edge-blurring drawbacks typical of linear spatial filters (such as Box or Gaussian blurs).

- **Median Filtering** is optimal for sparse salt-and-pepper impulse noise.
- **Alpha-Trimmed Mean Filtering** offers a balanced approach for images affected by mixed Gaussian and speckle noise.
- **Morphological Opening** provides precise removal of isolated bright artifacts.

Key Limitation: Non-linear filters depend heavily on appropriate kernel size selection. Increasing the window size beyond 3×3 frequently leads to detail loss, corner rounding, and structural blurring across all image types.

Github Link:- https://github.com/SlockAhuja/DSIP.git